

## PYETJE DHE PERGJIGJEJE PREJ PROVIMEVE

1. Si do te karakterizonit dallimet midis specifikimeve te kerkesave dhe aktiviteteve te dizajnit ne ciklin jetesor te zhvillimit te software?

Specifikimi i kerkesave karakterizohet si software product design ku specifikohen features aftesite dhe interfaces qe do te kete produkti softwareik ne menyre qe te permbush kerkesat e klientit, Solution e ka SRS-ne

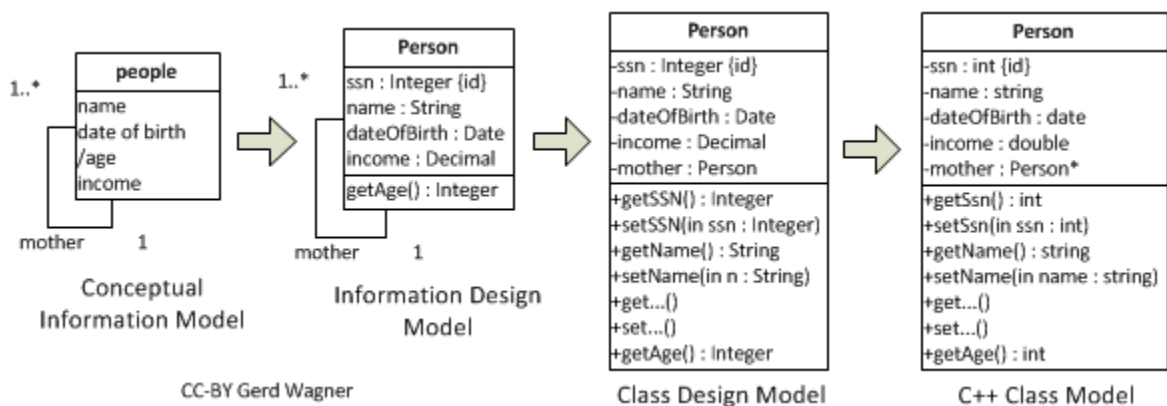
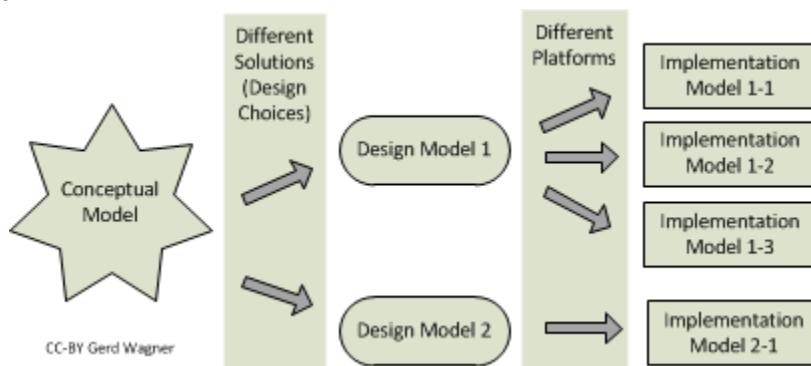
Kurse aktivitetet e dizajnit karakterizohen si enngenierring design ku specifikohen programet subsistemet dhe pjeset e tyre punuese per te permbush software product specifications. Solution e ka design document.

Te gjitha aktivitetet e dizajnit te ciklit jetesor te zhvillimit te softuerit bazohen ne kerkesat e specifikuara.

2. Jepni shembuj te dickaje qe nuk duhet te shfaqet ne nje model konceptual por mund te shfaqet ne nje model te klases se dizajnit:

Conceptual Model - important entities or concepts in the problem, atributet, important relationships

Design Class Model - klasat ne sistem softuerik, atributet, associations, operacionet por jo implementation details.



3. Cilat karakteristika duhet te kete nje paisje virtuale ideale?

1. SIMPLICITY - has simple, complete interface
2. COHESION - does exactly one job completely
3. COUPLING - it is loosely coupled to the rest of the program!
4. INFORMATION HIDING - Hides it's implementation
5. STABILITY - Never changes

4. Cili eshte dallimi mes kalimit te nje referimi si argument (pass by value) dhe kalimit te nje argumenti me reference (pass by reference) ? dhe avantazhet e tyre

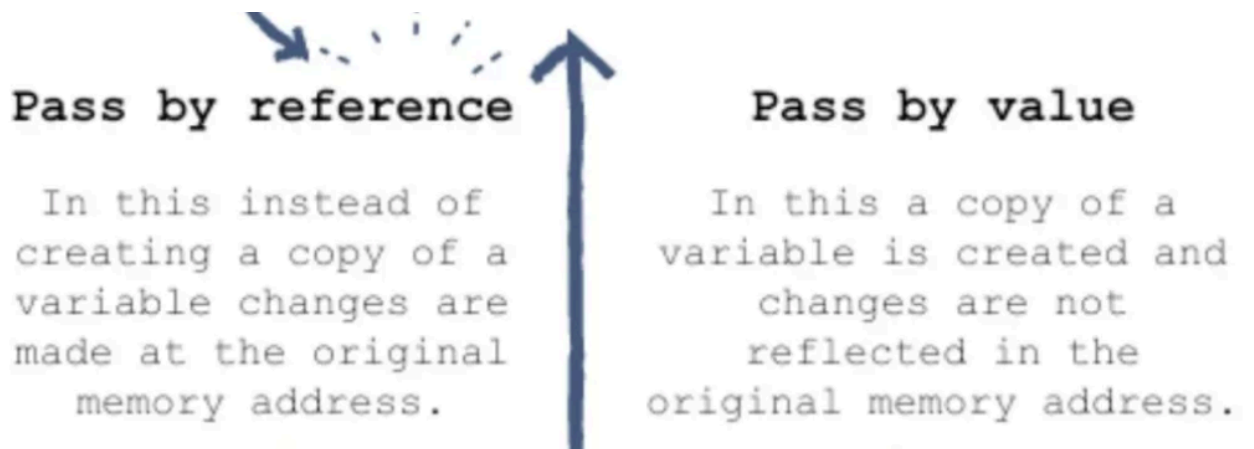
Tek pass by value you pass the value of the variable into the function (psh veq vleren 9 )  
kurse tek pass by reference you pass the variable into the function (ja dergon si ni integer me vlere 9)

The terms "pass by value" and "pass by reference" are used to describe how variables are passed on. To make it short: pass by value means the actual value is passed on. Pass by reference means a number (called an address) is passed on which defines where the value is stored.

To make a decision whether to use pass by reference or pass by value, there are two simple general rules:

If a function should return a single value: use pass by value

If a function should return two or more distinct values: use pass by reference



5. Si nderlidhet rifaktorimi i kodit me mostrat e dizajnit ose design patterns?

Rifaktorimi - proces sistematik i permiresimit te kodit pa krijuar funksionalitete te reja. Refaktorimi transformon kodin e "remujshem" ne te "regullt" dhe simple design.

Design Pattern - zgjidhje tipike per probleme te shpeshta ne dizajnin softuerik. Jan blueprints qe merren dhe personalizohen per me zgjidhe probeme specifike ne kod.

Nderlidhja e krijimit te procesit te permiresimit te dizajnit te kodit ekzistues me patterns, mundemi me rifaktoru duke perdor design patterns. Psh. factory method

Refaktorimi favorizon riperdorimin e elementeve te dizajnit (si design patterns) dhe code modules.

6. Si mund ta thjeshtesoj dokumentimin e dizajnit design pattern?

Streamlining documentation - pattern form and behavior need not be elaborated . Sigurojne zgjidhje gjenerale, dokumentohet ne nje format qe nuk kerkon specifika te lidhura per nje problemi specifik. Design patterns ndihmojne me shkru kod ma shpejt se ta ofron nje cleaner picture se qysh pe implementon ni dizjan.

7. Sqaroni parimin e funksionimin e Model View Controller architecture, MVC:

Modelon se si do te jene lidhjet mes user interface dhe problem-domain components.

Ky stil i arkitektures permban:

Model - problem-domain component me te dhena dhe operacione,

View- data display component

dhe Controller - component qe merr dhe vepron sipas user input.

Views edhe Controlles munden me u shtu, largu ose me u ndryshu pa e ngacmu

Modelin. Views munden me u shtu ose me u ndryshu edhe gjate ekzekutimit.

Por eshte e veshtire me i nda views edhe controllers, updates e shpeshta munden me e ngadalsu paraqitjen e te dhenave dhe keshtu performanca e user interface bjen.

Ky stil i bon user interface components shume te varur ne model components.

Si funksionon ne praktike eshte qe browser dergon request tek controler, controller ndervepron me modelin per me dergu dhe me pranu te dhena. Controller pastaj ndervepron me View per me i renderu te dhenat. View eshte e fokusuar vetem se si ti paraqes informacionet dhe jo prezentimin final.

8. Cila është diferenca mes percaktimit deklarativ dhe atij procedural të sjelljes së operacioneve?

Percaktimi deklarativ shpjegon të gjitha inputet outputet calling constraints dhe rezultatet pa e specifikuar algoritmin kurse ai procedural përshkruan algoritmin se si transformohen inputet në outputs ( algoritmi paraqet një sekuencë të hapave që mund të kryhen nga një kompjuter)

9. Cka eshte fshehja e informacionit? Ne cilat raste do te kishit tje kaluar vizibilitetin e entiteteve te programit

Fshehja e informacionit eshte kufizimi i qasjes ne entitetet e nje programi sa me shume qe mundemi.

Dy raste:

Kur moduleve iu nevojitet me nda nje entitet qe me mujt me bashkpunu psh a shared queue.

Kur nje synim tjetër i dizajnit eshte me me shume rendesi se fshehja e informacionit psh performance constraint.

10. Cilat modele (diagrame) mund te perdoren per te paraqitur kolaborimet ndermjet pjeseve te programit?

Sequence dhe Communications diagramet, activity diagramet, Box and line diagramet dhe use case modelet.

11. Cfare relacionit ekziston ndermjet coupling dhe cohesion?

Coupling eshte niveli i varshmerise se moduleve mes njeri tjetrit.

Cohesion eshte niveli ne te cilin pjeset e modulit jan te lidhura me njera tjetren.

Loose Coupling & High Cohesion

Te dyja keto jane principe qe zakonisht e mbeshtesin njera tjetren. Nje modul me high cohesion duhet te jete i pa varur nga modulet e tjera keshtu duke e zvogeluar coupling. Bashk keto formojne nje nder principet kryesore te nje arkitekture te mire.

12. Sqaro si e perkrah riperdorueshmerine stili i arkitektures layered:

Layers e meposhtem nuk kan varshmeri ndaj layers mbi, duke i mundesuar qe potencialisht te jene te riperdorshem ne scenarios te tjera.

Komunikimi mes layers eshte i bazuar ne abstraksion dhe evente per me siguru loose coupling mes layers.

13. Nje nga menytrat e gjenerimit te modeleve te nivelit te mesem eshte permes temave te dizajnit (design themes). Sqaro kete proces:

Procesi i gjenerimit te modeleve te nivelit te mesem permes design-themes fillon duke e krijuar nje design story e cila duhet te jete nje perskrim i shkurter i aplikacionit dhe e cila thekson pjeset me te rendesihme te tij. E analizon design story dhe permes sajje i indentifikon design themes, pastaj prej tyre nxjerr klasa kandidate duke ia percaktuar seciles pergjegjesite konkrete, ato qe kan pergjegjesi te ngjajshme ose dalin jasht scope te aplikacionit i largojme. Klasat e fituara nga lista i qesim ne class diagram.

14. Si arrihet high cohesion edhe low coupling?

High cohesion arrihet kur klasat brenda nje moduli kryejne veprime te njejta dhe bashkeveprojne me shume me njera tjetren sesa me klasat e moduleve te tjera. Low coupling kur kemi sa me pak nderveprim mes (klasave te moduleve te ndryshme.

15. A mjafton mikroserviset me pas vec API gateway per komunikim a vyn edhe dicka tjeter?

16. Product design Process edhe Engineering design process behen ne cilin proces dhe tregoni te pakten nje arsye pse eshte i dobishem:

Keto dyja behen ne Generic Software Product Design Process. Product Design Procces eshte i dobishem pasi me ane te tij gjejme SRS kurse me ane te Engineering design process gjejme Design Docment

17. A jane njelloj produkti softuerik dhe produkti me softuer ne te ?

Produkti me softuer ne te psh bankomati ose kerri..... E softueret jane wordi excel e qito

18. Parakushtet dhe paskushtet te specifikimi i operacioneve, merr shembull
- Precondition is an assertion that must be true at the initiation of an operation.
- Postcondition is an assertion that must be true upon completion of an operation.

Shembull:

|                    |                                                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Signature</b>   | public static int findMax( int[] a ) throws<br>IllegalArgumentException                                                                                     |
| <b>Class</b>       | Utility                                                                                                                                                     |
| <b>Description</b> | Return one of the largest elements in an int array.                                                                                                         |
| <b>Behavior</b>    | pre: (a != null) && (0 < a.length)<br>post: for every element x of a, x <= result<br>post: throws IllegalArgumentException if<br>preconditions are violated |

19. Si finalizohet arkitektura softuerike? qysh e dijm qe u kry, cka duhet me plotesu ne fund qaj sen ?
20. A SAD should be reviewed when substantially complete to ensure that it
- Specifies a feasible and adequate architecture;
  - Has well-formed models; and
  - Is complete, clear, and consistent.
- Various sorts of reviews can be used.
  - An effective form of review is the active review.
  - Reviews should be done when artifacts are complete and also throughout the design process.

21. Perpos Box&Line diagram cilat tjera notacione perdoren edhe per cka perdoren ne DeSCRIPTR

| Type of Specification | Useful Notations                                                                                 |
|-----------------------|--------------------------------------------------------------------------------------------------|
| Decomposition         | Box-and-line diagrams, class diagrams, package diagrams, component diagrams, deployment diagrams |
| States                | State diagrams                                                                                   |
| Collaborations        | Sequence and communication diagrams, activity diagrams, box-and-line diagrams, use case models   |
| Responsibilities      | Text, box-and-line diagrams, class diagrams                                                      |
| Interfaces            | Text, class diagrams                                                                             |
| Properties            | Text                                                                                             |
| Transitions           | State diagrams                                                                                   |
| Relationships         | Box-and-line diagrams, component diagrams, class diagrams, deployment diagrams, text             |

22. Nje nga principet baze te dizajnit eshte ndryshueshmeria (Changeability). Pse eshte ajo aq e rendesishme?

Dizajnet qe bejne programe qe jane te lehta per tu ndryshuar jane me te mira. Ndryshueshmeria eshte nje aspekt i rendesishem sidomos ne ambiente ku softuerit i kerkohen ndryshime te vazhdueshme. Nese me vone edhe pas zhvillimit te produktit kerkohen ndryshime nga klienti ose edhe gjenden gabime gjate procesit, kur softueri eshte lehte i ndryshueshem kjo na kursen neve ne kohe, para dhe resurse njerezore.

23. Shpjegoni stilin e akitektures layered

Layered architecture eshte nje prej stileve te arkitektures me te perdorua. Programi ndahet ne nje varg layeres, Layers (modulet) i perdorin sherbimet e layer/s qe ndodhen me poshte dhe iu ofrojne sherbime laryer/s qe jane permby. Cdo program unit duhet me qene saktesisht nje layer. Ndahet ne strukture statike dhe dinamike. Statikja - software ndahet ne layers ku secili ofron nje set sherbimesh me interface te percaktuar mire. Dinamike- co layer eshte i lejuar me perdor veq layer posht tij (Strict layered) ose krejt posht tij (relaxed layered)



24. Cila eshte ma abstrakte design principles apo design patterns, jepni shembuj prej te dyjave?

Prej google—

Design principles, on the other hand, are general guidelines to help create robust designs .Design principles provide high level guidelines to design better software applications. They do not provide implementation guidelines and are not bound to any programming language. The SOLID (SRP, OCP, LSP, ISP, DIP) principles are one of the most popular sets of design principles. For example, the Single Responsibility Principle (SRP) suggests that a class should have only one reason to change. This is a high-level statement which we can keep in mind while designing or creating classes for our application. SRP does not provide specific implementation steps but it's up to you how you implement SRP in your application.

Design patterns are typical ways to organise the components of a program in typical situations. Design Pattern provides low-level solutions related to implementation, of commonly occurring object-oriented problems. In other words, design pattern suggests a specific implementation for the specific object-oriented programming problem. For example, if you want to create a class that can only have one object at a time, then you can use the Singleton design pattern which suggests the best way to create a class that can only have one object.

Prej slides—

Design principles jane statements te cilat tregojne se cfare e bene nje dizajne me te mire. Kemi dy lloje Basic Principles te cilat tregojne(state) karakteristikat te cilat e bejne nje dizajne me te mire qe te permbushin nevojat dhe deshirat e stakeholders. Constructive principles konstatojne “bazuar ne eksperience” se disa engineering design characteristics e bejne nje dizajn me te mire .

Basic Principles

- Feasibility • A design is acceptable only if it can be realized.
- Adequacy • Designs that meet more stakeholder needs and desires, subject to constraints, are better.
- Economy • Design that can be built for less human efforts, less cost, in less time, with less risk, are better.
- Changeability • Design that make a program easier to change are better

Constructive Principles

- Modularity principles • Good design are modular; these principles help evaluate whether designs specify good modules.
- Implementability principles • Good designs are easier to build; these principles help evaluate whether designs will be easy to implement.
- Aesthetic principles • Good designs are beautiful; these principles help pick out beautiful designs.

25. Dallimi ndermjet analizes dhe rezolucionit te dizajnit.

26. Cka jane Enkapsulimi, Polimofrizmi, Defensive Copy, dhe Overloading

Enkapsulimi perdoret per te fshehur strukturën e të dhënave brenda klasës, duke parandaluar palët e pa autorizuar të kenë qasje direkte në to. Ai perdoret per tiu referuar dy nocioneve te lidhura mes veti por te cilat dallojne nga njera tjera dhe ndonjehere kombinimit te tyre:

- nje mekanizem i gjuhes i cili kufizon qasjen ne disa komponente te objekteve
- nje konstrukt i gjuhes qe mundeson mbledhjen e te dhenave dhe metodave qe veprojne mbi ato te dhena

Polimorfizmi eshte mundesia e objektit qe te merr shume forma. Ndodhe kur kemi shume klasa qe kane te bejne me njera-tjetren nga trashegimia, pershkruan veqorite e gjuheve qe lejojne te njejten fjale ose simbol te interpretohet korrekt ne situatë te ndryshme bazuar ne kuptimin e saj. Fjale per fjale polimorfizem do te thote “ shume forma ” – dhe pershkruan vecorine e gjuheve qe lejojne te njejten fjale ose simbol te interpretohet korrekt ne situata te ndryshme bazuar ne kuptimin e saj. Psh: Fjala “run” interpretohet ndryshe ne sport, biznes, programim.

Defensive Copy eshte nje teknike e cila zbut efektet negative te shkaktuara nga modifikimet e paqellimshme(ose te qellimshme) te objekteve te perbawshketa. Sic tregon vet emri, ne vend qe te ndajme objektin original, ne ndajme nje kopje te tij dhe keshtu cdo modifikim i vere ne kopje nuk do te ndikojë ne objektin original.

- A defensive copy is a copy of an entity held by reference passed to or returned from another operation.

Overloading lejon qe metoda te ndryshme te kene te njejtin emer, por nenshkrimeve te ndryshme ku nenshkrimi mund te ndryshojë nga numri i parametrave te hyrjes ose lloji i parametrave te hyrjes.

Overloading mundeson krijimin brenda nje klase te disa metodave me te njejtin emer te cilat ndryshojne vetem ne numrin dhe llojin e input-parametrave dhe vleres kthyes te metodës

27. Trego disa ceshtje specifike te dizajnnuesve te produktit softuerik dhe inxh softuerike. Dizajnuesit e produktit softuerik mirren me stilimin, estetiken funksionin dhe perdroshmerine kurse dizajnuesit e inxh softuerike merren me mekanizmat dhe funksionet teknike

28. Dallimi ne mes class diagramit dhe object diagramit?

Class diagramet tregojne klasat dhe lidhjet mes tyre, cka perbajne objektet ne sistemin tuaj dhe cfare jane ne gjendje te bejne ato (metodat ) kurse

Object diagramet reprezentojne nderveprimin e objekteve ne nje pike gjate ekzekutimit

29. Sqaroni dallimin ndermjet analizes dhe rezulucionit te dizajnit?

Analiza eshte ndarja e problemit te dizajnit per ta kuptuar ate kurse rezulucioni eshte zgjidhja e nje problemi te dizajnit .

Design = Analysis + Resolution

30. Cka eshte design theme?

A design theme is an important problem, concern, or issue that must be addressed in a design. Design themes can be the basis for generating a design from scratch.

31. Cka eshte stili arkitektonik?

An architectural styles is a high-level model for entire programs and sub-systems.

32. Event driven a mban te dhena dhe sqaro?

Jo? Veq i transferon te dhenat nepermes decoupled microservices

33. Lidhja ne mes data type dhe cohesion ?

Ne baze te nje personi qe ka shkruar pergjigje ka thene keshtu:

Kohezioni eshte me i larte ne modulet qe kane role te pastert, logjik dhe pergjegjesi te pavarur. Nje zgjidhje praktike dhe long-standing per te arritur kohezion eshte te formohen module qe implementojne datatypes.

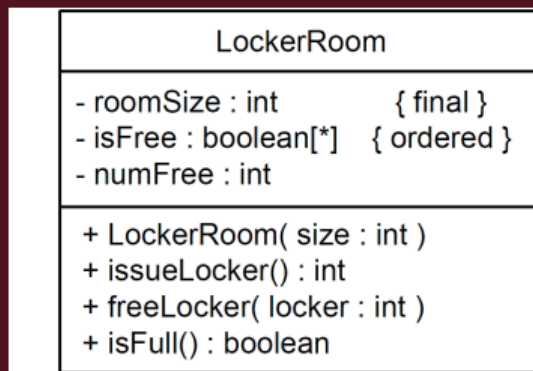
Data type ka dy elemente: set te vlerave dhe koleksion te operacioneve per manipulim dhe ekzaminim te vlerave ne carrier set.

34. Cka jan design pattern dhe si karakterizohen sipas Gang of Four

A software design pattern is a model proposed for imitation in solving a software design problem.

35. Cka jan invariantet e klases, jep shembull

A class invariant is an assertion that must be true of any class instance between calls of its exported operations.



**Figure 14-2-2 The LockerRoom Class**

- `roomSize` must be at least 1.
- `numFree` attribute must vary between 0 and `roomSize`.
- the number of true values in the `isFree` array must be `numFree`.

inv:  $0 < \text{roomSize}$  inv:  $0 \leq \text{numFree} \ \&\& \ \text{numFree} \leq \text{roomSize}$  inv: `numFree` is the number of true elements in `isFree`

36. Pse duhet me u rishiku/review dizajni? Cka osht Critical Reviews Process

Dizajni duhet me u rishiku sepse eshte e mundur te jene leshuar gabime si ne pjesen e dizajnit si ne pjesen e implementimit. Rishikohet nese i ploteson te gjitha kerkesat. Eshte shume e dobishme nese behen reviews ne menyre te vazhdueshme gjate te gjith procesit te krijimit te produktit softuerik sepse nese ka gabime ato eliminohen menjehere. Nese gabimet gjenden pas perfundimit te te gjitha fazave kjo na kushton duke na kthyer mbrapa tek fazat fillestare te dizajnit na kushton shume me shtrenjte, na merr shume kohe dhe eshte frustruese.

A critical review is an evaluation of a finished product to determine whether it is of acceptable quality

37. Disa hapa qe duhet me i ndjek per me kthi ni model konceptual ne model mid level

- Change actors to interface classes.
- Add actor domain classes.
- Add a startup class.
- Convert or add controllers and coordinators.
- Add classes for data types.
- Convert or add container classes.
- Convert or add engineering design associations.

38. Cka duhet me pas ni dokument i dizajnit

A design document consists of a SAD and a detailed design document (DDD).  
A DDD template.

1. Mid-Level Design Models
2. Low-Level Design Models
3. Mapping Between Models
4. Detailed Design Rationale
5. Glossary

39. Ndertimi i arkitektures se softuerit eshte nje proces shume iterativ i perseritur, Pse?

Dizajneret duhet qe shume shpesh te rianalizojne problemin dhe te gjenerojne dhe permiresojne zgjidhjet shume here. Pothuajse cdo proces gjate ndertimit te akitektures se softuerit kalon ne fazat e Analizimi te problemit, gjenerimit te zgjidhjeve, evaluimit te ketyre zgjidhjeve, zgjedhja e tyre nese nuk eshte e duhura e njejta perseritet prap. Cdo proces ka te njejten procedure. Analizo, Gjenero, Evaluo, Select, Finalize.

40. Si ndikojne kompanite e zhvillimit ne dizajin e arkitektures dhe si ndikojne arkitekturat e softuerit ne kompanite e zhvillimit

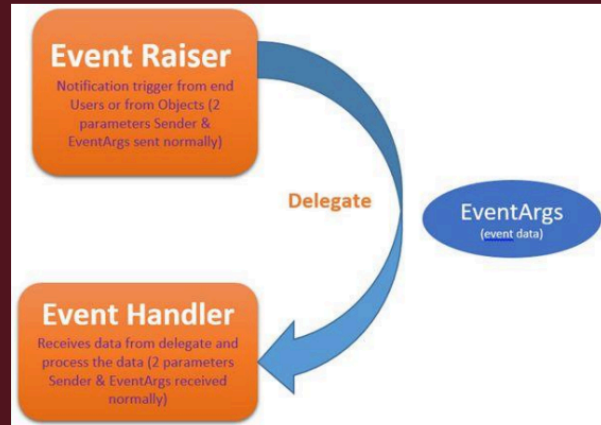
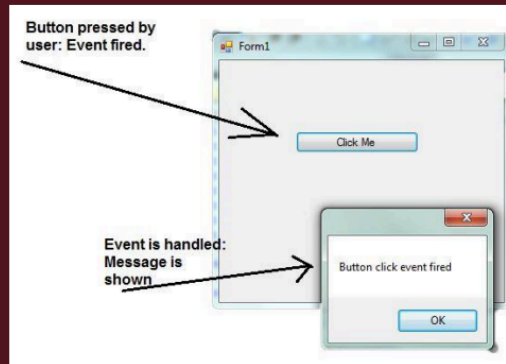
41. Cka eshte RIP, shkruaj nje shembull.

The Replace of IFs with Polymorphism is known as RIP design pattern

42. Cka eshte delegation dhe jep shembull?

Delegation Delegation is a tactic wherein one module (the delagator) entrusts another module (the delegate) with a responsibility. • Allows reuse without violating inheritance constraints • Makes software more reusable and configurable

## Delegation Example



36

### 43. Component diagram vs deployment diagram?

- A UML component is a modular, replaceable unit with welldefined interfaces.
- Component symbols are rectangles containing names
- Stereotyped «component» or have component symbol in upper right-hand corner
- A UML component diagram shows components, their relationships to their environment, and their internal structure.

A UML deployment diagram models computational resources, communication paths among them, and artifacts that reside and execute on them.

- Used to show
- Real and virtual machines used in a system
- Communication paths between machines
- Program and data files realizing the system
- Residence Execution

### 44. Dallimi mes static and dynamic models?

A static model represents aspects of programs that do not change during program execution. A dynamic model represents what happens during program execution. Static model examples include class and object models.

- Dynamic model examples include state diagrams and sequence diagrams.

### 45. Java interfaces, Uml interfaces dhe Module interinterfaces?

Interfaces are very important in component diagrams because they define the relationship between a component and its environment

- A UML interface is a named collection of public attributes and abstract operations.
- Represented by a stereotyped class symbol (later)
- Represented by special ball and socket symbols

In Java, an interface is **an abstract type that contains a collection of methods and constant variables**. It is one of the core concepts in Java and is used to achieve abstraction, polymorphism and multiple inheritances. We can implement an interface in a Java class by using the implements keyword

Module Interfaces means **any interaction protocol that can be used between two Object Code Modules**, including without limitation (i) any function call protocol, including the explicit description of all input and output parameters, their meaning and authorized values, (ii) the definition of any data structure including .

46. Design rationale?

Architectural Design Rationale — Explanation of difficult, crucial, puzzling, and hard-to-change design decisions